

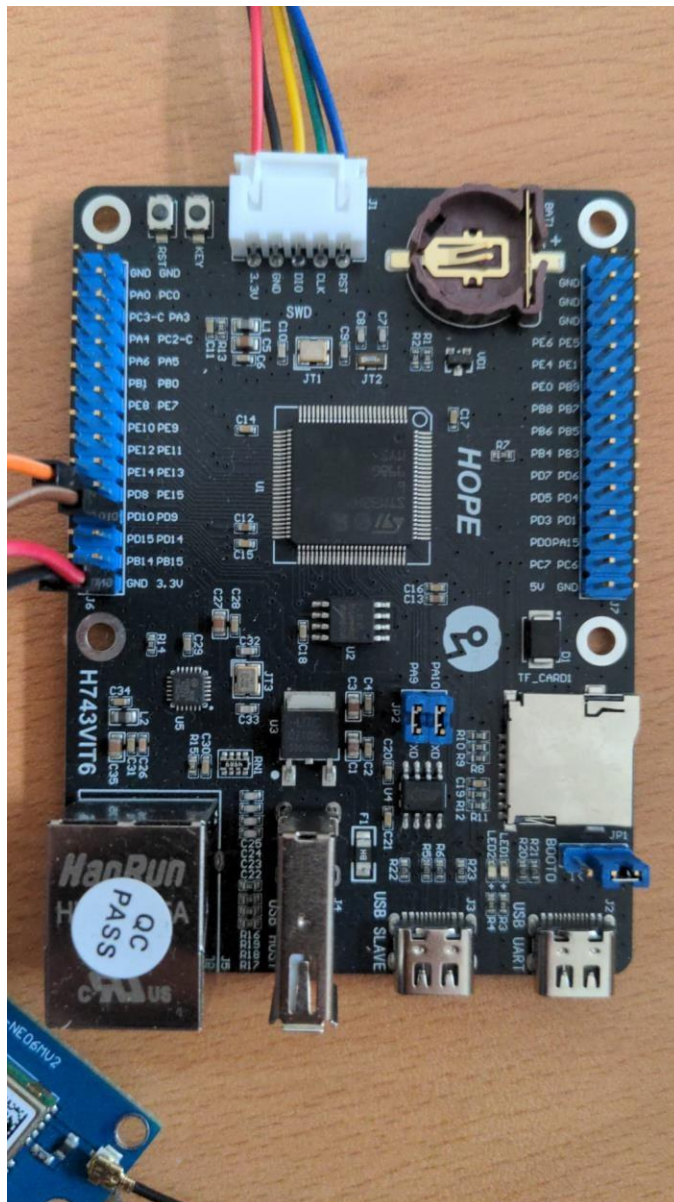
- **1. Project Idea and Objectives**

- **1.1. The Project Idea** The main idea of this project is to build a basic location-tracking embedded system using an STM32H7 microcontroller and a GPS module. Simply put, the hardware setup receives signals from satellites, and the microcontroller processes these signals to determine the current geographical location (such as my current coordinates here in Štip).
- As a computer engineering student, I chose this project to gain hands on experience with embedded systems. It provides a great opportunity to move beyond theoretical software concepts and work directly with physical hardware components.
- **1.2. Project Objectives** To complete this project successfully, I established the following clear objectives:
 - **Hardware Setup:** To correctly wire and integrate the core components: the STM32H7 development board, the GPS receiver module, and the ST-Link programmer.
 - **Establishing Communication:** To ensure that the microcontroller can successfully receive the data stream sent by the GPS module.
 - **Data Processing:** To write a functional C program that reads the incoming raw text from the GPS and cleanly extracts the specific location coordinates into variables.
 - **Problem Solving and Debugging:** To overcome the hardware limitations and debugging issues I encountered (specifically those related to using a clone ST-Link device) and to find practical workarounds to ensure the firmware works and can be verified properly.

2. Description of the Hardware Components

This section outlines the primary hardware components used to build the physical setup of the project. To ensure clear communication and reliable debugging, specific pins and interfaces were carefully selected and wired.

2.1. Microcontroller: STM32H743VIT6 Development Board



The brain of this project is an STM32H743VIT6 development board. In an embedded system, the microcontroller acts as the central processing unit; it executes the C firmware, manages memory, and controls the flow of data between different peripherals.

Pinout diagram of the STM32H743VITx LQFP100 package. The diagram shows the 100 pins of the package with their functions. The top row includes VDD, VSS, PE1, PE0, PB9, PB8, BOOT0, PB7, PB6, PB5, PB4, PB3, PD7, PD6, PD5, PD4, PD3, PD2, PD1, PD0, PC12, PC11, PC10, PA15, and PA14. The right side includes VDD, VSS, VCAP, PA13, PA12, PA11, PA10, PA9, PA8, PC9, PC8, PC7, PC6, PD15, PD14, PD13, PD12, PD11, PD10, PD9, PD8, PB15, PB14, PB13, and PB12. The bottom row includes VSS, VDD, PA4, PA5, PA6, PA7, PC4, PC5, PC6, PC8, PB1, PB2, PE7, PE8, PE9, PE10, PE11, PE12, PE13, PE14, PE15, PB10, PB11, VCAP, VSS, and VDD. The left side includes PE2, PE3, PE4, PE5, PE6, VBAT, PC13, PC14, PC15, VSS, VDD, PH0, PH1, NRST, PC0, PC1, PC2, PC3, VSSA, VRE, VDDA, PA0, PA1, PA2, and PA3. The center of the package features the ST logo and the text 'STM32H743VITx LQFP100'. External connections are shown for RCC_OSC_IN, RCC_OSC_OUT, DEBUG_JTMS-SWDIO, USART3_RX, and USART3_TX.

- ## 2.2. GPS Receiver: NEO-6M

To acquire the satellite data, I integrated the GY-NEO6MV2 module, which features the standard NEO-6M GPS chip attached to a ceramic antenna. At a basic functional level, a GPS receiver works by catching radio frequency signals from multiple orbiting satellites. By calculating the exact time it takes for these signals to arrive, the module determines its precise global position and continuously outputs this information as standard text strings. This module communicates via the UART protocol at a default baud rate of 9600.



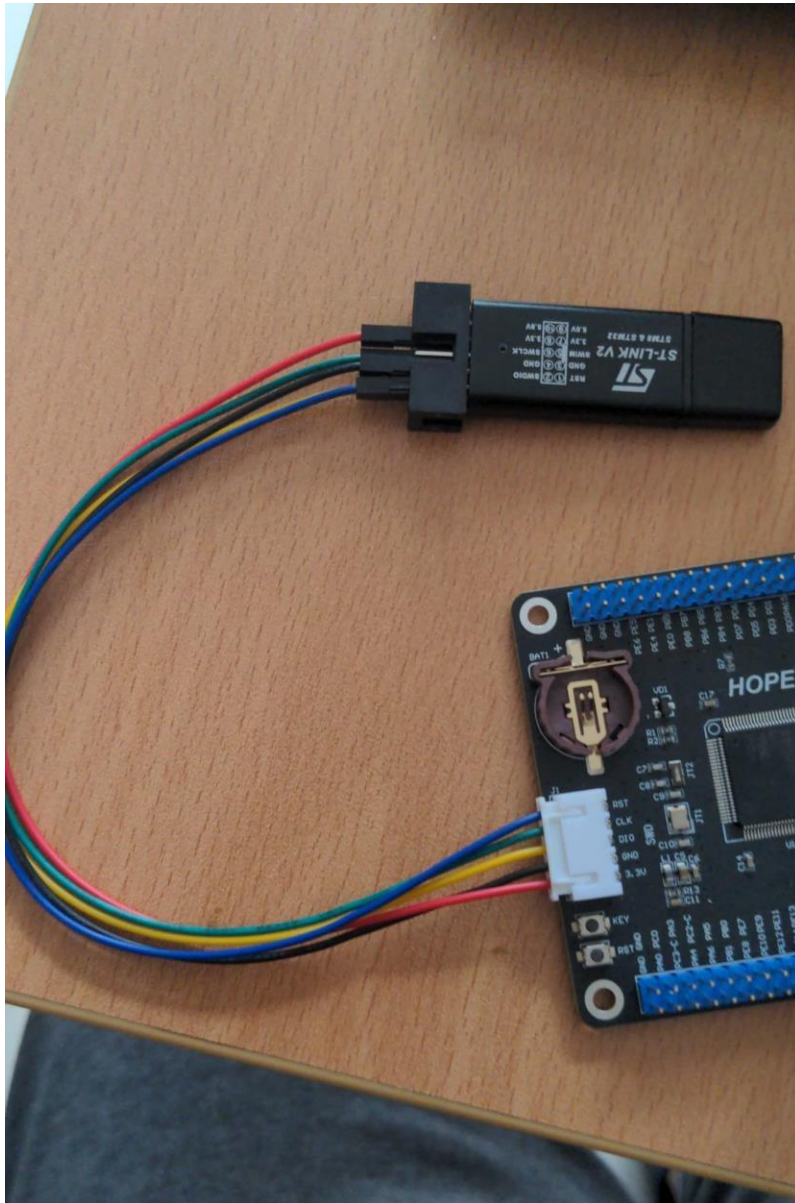
The wiring between the NEO-6M and the STM32 was established as follows:

- **VCC:** Connected to the 3.3V power supply.
- **GND:** Connected to the common Ground.
- **TX (GPS):** Connected to the STM32's **PD9** (USART3_RX) to send the NMEA sentences to the microcontroller.
- **RX (GPS):** Connected to the STM32's **PD8** (USART3_TX).

2.3. Debugger and Programmer: ST-Link V2 (Clone)

To program the microcontroller and debug the code, I used a USB-dongle style ST-Link V2. A debugger essentially acts as a hardware bridge between the computer and the microcontroller. It takes the compiled code from the PC via USB and translates it into specific electrical signals to flash the microcontroller's memory, while also

allowing the developer to monitor variables in real-time. While this is a clone version of the original STMicroelectronics hardware, it provides the necessary SWD (Serial Wire Debug) interface to flash the firmware.

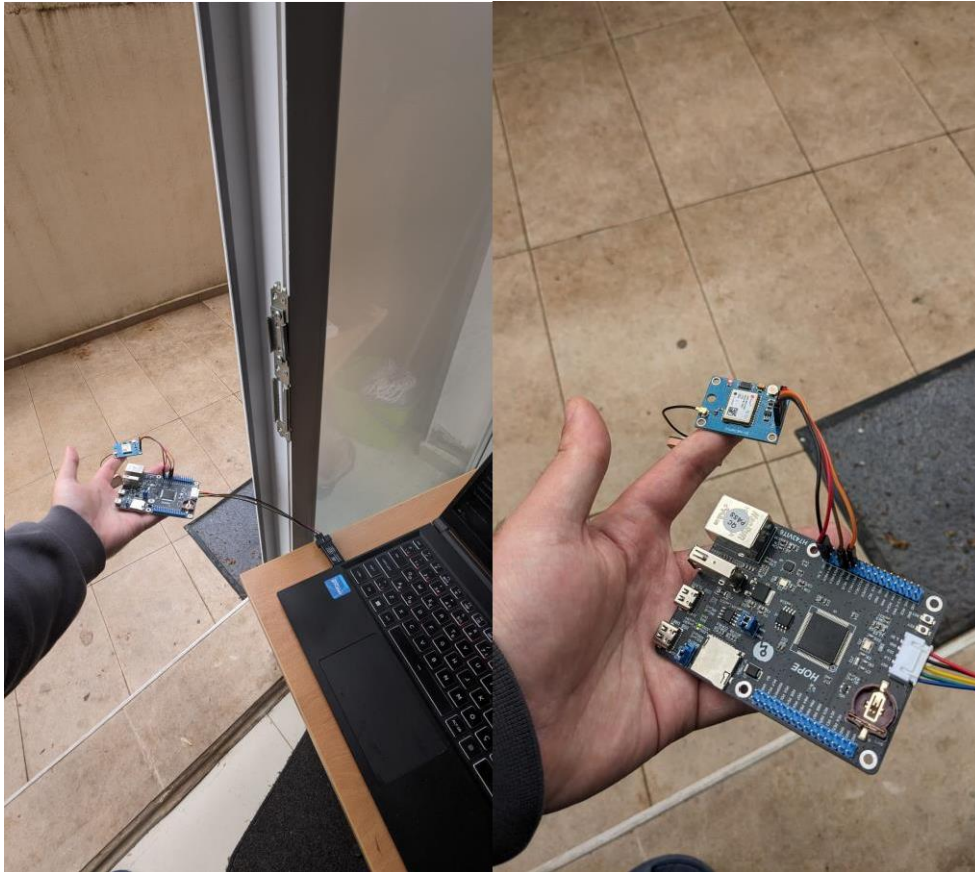


The ST-Link is connected directly to the dedicated SWD header on the STM32 board using the following pinout:

- **3.3V:** Power reference
- **GND:** Common ground
- **SWDIO (DIO):** Serial Wire Data Input/Output for data transfer
- **SWCLK (CLK):** Serial Wire Clock for synchronization
- **NRST (RST):** Hardware Reset pin

2.4. Complete Hardware Setup and Field Testing

Since GPS modules require a clear line of sight to the sky to lock onto satellites and retrieve valid \$GPGGA data, indoor testing is often unfeasible. Therefore, the complete hardware setup (Microcontroller + GPS + ST-Link connected to the laptop) had to be positioned outside during the testing and data acquisition phases to ensure reliable signal reception.



3. Explanation of the Implementation

This section details the software architecture of the project. The firmware was written in C using the STM32 HAL (Hardware Abstraction Layer) libraries. The core logic is divided into two main parts: asynchronous data acquisition using a Ring Buffer and real-time data parsing.

3.1. Initialization and UART Configuration Upon system startup, after initializing the System Clock and the MPU (Memory Protection Unit), the USART3 peripheral is initialized. Since the GPS module outputs data continuously at a fixed rate, USART3 is configured with a baud rate of 9600.

Instead of using standard polling methods (which halt the CPU until data arrives), I initialized an interrupt-driven background process using the `Ringbuf_init()` function.

3.2. Data Acquisition: The Ring Buffer Architecture One of the major challenges in reading continuous GPS data is preventing data loss (buffer overrun). The GPS module does not wait for the microcontroller to be ready; it sends characters constantly. If the microcontroller is busy executing a delay or parsing a previous sentence, incoming characters will be lost.

To solve this, I implemented a Ring Buffer. When `Ringbuf_init()` is called, it enables the `UART_IT_RXNE` (Receive Data Register Not Empty) interrupt. Every time a new character arrives from the GPS, the hardware triggers the `Uart_isr` (Interrupt Service Routine). This routine automatically reads the character from the data register (RDR) and saves it to an array (`_rx_buffer`) in the background, independently of the main program loop.

3.3. Filtering NMEA Sentences The GPS module transmits various NMEA sentences (e.g., `$GPGLL`, `$GPGSA`, `$GPGSV`), but for this project, I only needed the `$GPGGA` sentence, which contains the essential 3D location and accuracy data.

In the infinite `while(1)` loop, I used the custom `Wait_for("GGA")` function. This function continuously peeks into the Ring Buffer without halting the whole system. It scans the incoming character stream until it detects the specific sequence "GGA".

```
while (1)
{
    if (Wait_for("GGA") == 1)
    {
        Copy_upto(";", gps_ham_veri);
    }
}
```

Once the "GGA" header is confirmed, the `Copy_upto("*", gps_ham_veri)` function extracts the rest of the sentence from the circular buffer and stores it in a temporary character array named `gps_ham_veri` for safe processing.

3.4. Data Parsing Algorithm After isolating the raw GPGLA sentence, the data needs to be broken down into individual variables (time, latitude, longitude). The standard NMEA format separates data fields with commas (,).

I wrote a parsing algorithm called `get_nmea_field()`.

```
void get_nmea_field(char *nmea_str, int field_num, char *output) {
    int comma_count = 0;
    int i = 0, j = 0;

    while (nmea_str[i] != '\0' && nmea_str[i] != '*') {
        if (nmea_str[i] == ',') {
            comma_count++;
            i++;
            continue;
        }

        if (comma_count == field_num) {
            output[j++] = nmea_str[i];
        }

        if (comma_count > field_num) {
            break;
        }
        i++;
    }
    output[j] = '\0';
}
```

This function takes the raw sentence, counts the commas as it iterates through the characters, and extracts the exact string between the targeted commas. Using this logic, I systematically populated my global variables inside the main loop:

```
get_nmea_field(gps_ham_veri, 1, time);
get_nmea_field(gps_ham_veri, 2, latitude);
get_nmea_field(gps_ham_veri, 3, latitude_direction);
get_nmea_field(gps_ham_veri, 4, longitude);
get_nmea_field(gps_ham_veri, 5, longitude_direction);
```

Through this implementation, the microcontroller efficiently juggles the constant stream of satellite data in the background while precisely picking out the required coordinates in the foreground loop.

4. Challenges and Troubleshooting (The Debugging Process)

The most significant bottleneck in this project was not writing the C code, but rather dealing with the severe hardware and synchronization limitations of the clone ST-Link V2 debugger. For the first three to four days, spending 6 to 7 hours daily, I was entirely stuck on getting the microcontroller to communicate with the IDE properly. Below is a chronological breakdown of the major issues encountered and the solutions I implemented.

4.1. "Failed to Execute MI Command" and vFlashErase Errors Initially, when I tried to flash the code onto the STM32H7, the default GDB server failed completely. I switched the debugger setting to use **OpenOCD**, but then I immediately encountered persistent "Failed to execute MI command" and "vFlashErase" errors.

- **Attempted Solutions:** At first, I tried to bypass this by manually pressing the physical RST (Reset) button on the STM32 board at the exact millisecond the IDE attempted to flash. This manual timing trick worked only once and never again.
- **The Fix:** I realized that the physical RST pin was not wired between the ST-Link and the STM32. After making the physical connection, I researched the STM32 community forums and StackOverflow. Based on the advice found there, I changed the reset mode to "**Software System Reset**" in the debugger configuration and lowered the SWD frequency (e.g., to 4MHz). But 4MHz didn't work on my ST-Link so I lowered to 480KHz. This successfully eliminated the MI command and flash erase errors.

4.2. ST-Link Synchronization and Firmware Updates Even after the flashing errors were resolved, the connection was highly unstable. The debugger would frequently lose sync with the ARM core.

- **Attempted Solutions:** I tried to connect via STM32CubeProgrammer to perform a full flash erase, but the ST-Link refused to connect. I realized the clone ST-Link itself might have outdated firmware. It took 4 to 5 repeated attempts (and several manual resets) before the ST-Link firmware update tool finally managed to push the update to the dongle.
- **The Fix:** To further stabilize the OpenOCD connection, I added the following custom initialization commands to the startup script in the IDE: `monitor reset` `monitor halt` `monitor gdb_sync` This forced the debugger to properly halt and sync with the microcontroller before executing code. Crucially, it was only after successfully forcing the ST-Link firmware update that the system became stable enough to eventually receive real GPS data.

4.3. The "Live Expression" Trap (Memory Access Faults) The most time-consuming trap of the project occurred during data verification. To test my parsing logic, I initially injected a "mocked" (fake) NMEA string into the code. When I ran the debugger, I constantly monitored the "Live Expressions" and "Variables" tabs in the IDE. Both were completely empty. I assumed my code was fundamentally broken, which led me to scrap the project on the third day and recreate the entire CubeMX initialization from scratch.

- **The Breakthrough:** After days of frustration, I randomly decided to check the standard "Expressions" tab instead of the "Live" ones. To my surprise, the mocked data was perfectly parsed and visible there.
- **Conclusion:** I discovered that the clone ST-Link simply does not have the hardware capability to perform real-time memory access. It can only read the memory when the code is halted.

4.4. Final GPS Integration and Ring Buffer Once I understood how to properly read the memory using the standard Expressions tab, I removed the mocked data and proceeded to field testing.

- I took the NEO-6M GPS module outside and waited approximately 4-5 minutes until the red LED began blinking, indicating a successful satellite lock.
- I connected it to the STM32. Initially, standard UART polling dropped characters. After researching tutorial videos, I implemented the **Ring Buffer** architecture to handle the asynchronous nature of the GPS data.
- By combining the updated ST-Link firmware, the monitor startup commands, the Ring Buffer, and observing the data only through the static "Expressions" tab using breakpoints, I finally achieved a stable system that accurately displayed the parsed geographical coordinates.

4.5. Data Visualization and the Search for a Virtual COM Port (VCP) Even after successfully capturing and parsing the GPS data, I faced a final challenge regarding data visualization. When viewing the populated latitude and longitude character arrays in the IDE's "Expressions" tab, the formatting was quite cumbersome. The characters were displayed across multiple individual lines, forcing me to expand the IDE's "Details" pane just to read the complete coordinate strings.

To create a cleaner, more professional output, my immediate instinct was to print the parsed data to a serial terminal emulator on my PC using `printf` statements. However, I discovered that the clone ST-Link V2 USB dongle lacks the internal UART-to-USB

bridge required for a Virtual COM Port (VCP). It does not enumerate as a COM port on the computer, making direct serial output impossible through the debugger.

To bypass this hardware limitation, I realized I needed an external USB-to-TTL serial converter. I took a break from the software and went out to search for this physical component locally. I visited about four or five different electronics shops.

Unfortunately, local availability proved to be a significant constraint. At the final shop, the technician informed me that such specific embedded development modules are not sold in Stip, and my only options were to order it online or travel to Skopje to find one.

Given the time constraints of the project and the urgent need to resolve the core code issues first, I abandoned the search for the external converter. Consequently, I adapted to the situation, accepting the visual constraints of the IDE and relying exclusively on the static "Expressions" tab with breakpoints to verify the final results.

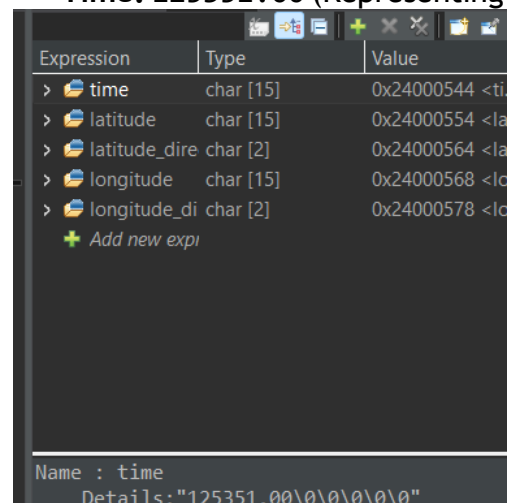
5. Results and Current State of the Project

5.1. Successful Data Acquisition Despite the significant hardware limitations and the lack of a Virtual COM Port for easy terminal debugging, the core objective of the project was successfully achieved. By utilizing the background Ring Buffer for stable UART reception and the custom string-parsing algorithm, the STM32H7 microcontroller reliably extracted the necessary location data from the NEO-6M GPS module.

5.2. Verifying the Coordinates

The system correctly identified the \$GPGGA sentence and split the comma-separated values into their respective arrays. As seen in the attached IDE screenshots, the "Details" pane reveals the following extracted NMEA strings:

- **Time:** 125351.00 (Representing 12:53:51 UTC time)



Latitude: 4144.26291 and Latitude Direction: N

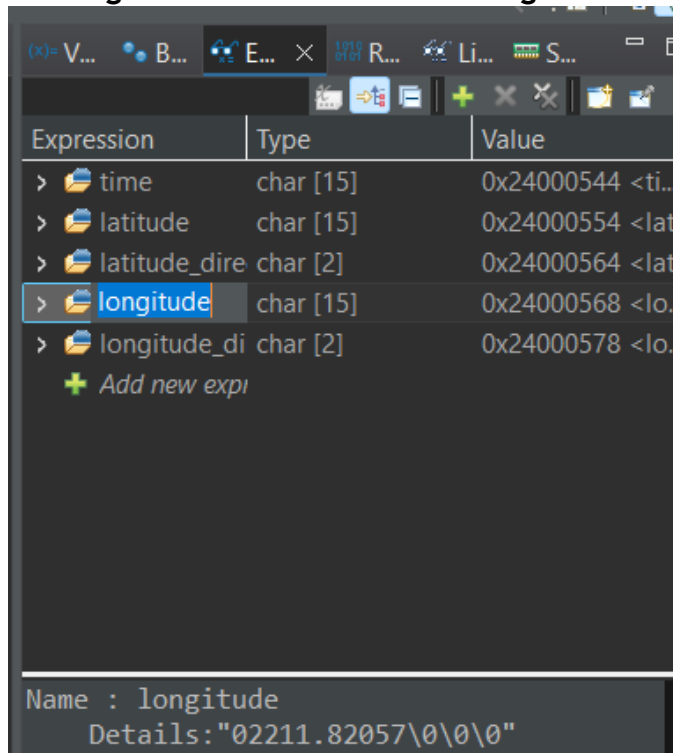
Expression	Type	Value
> time	char [15]	0x24000544 <ti...
> latitude	char [15]	0x24000554 <lat...
> latitude_dire	char [2]	0x24000564 <lat...
> longitude	char [15]	0x24000568 <lo...
> longitude_di	char [2]	0x24000578 <lo...
+ Add new expi		

Name : latitude
Details: "4144.26291\0\0\0\0"

Expression	Type	Value
> time	char [15]	0x24000544 <ti...
> latitude	char [15]	0x24000554 <lat...
> latitude_dire	char [2]	0x24000564 <lat...
> longitude	char [15]	0x24000568 <lo...
> longitude_di	char [2]	0x24000578 <lo...
+ Add new expi		

Name : latitude_direction
Details: "N"

Longitude: 02211.82057 and Longitude Direction: E

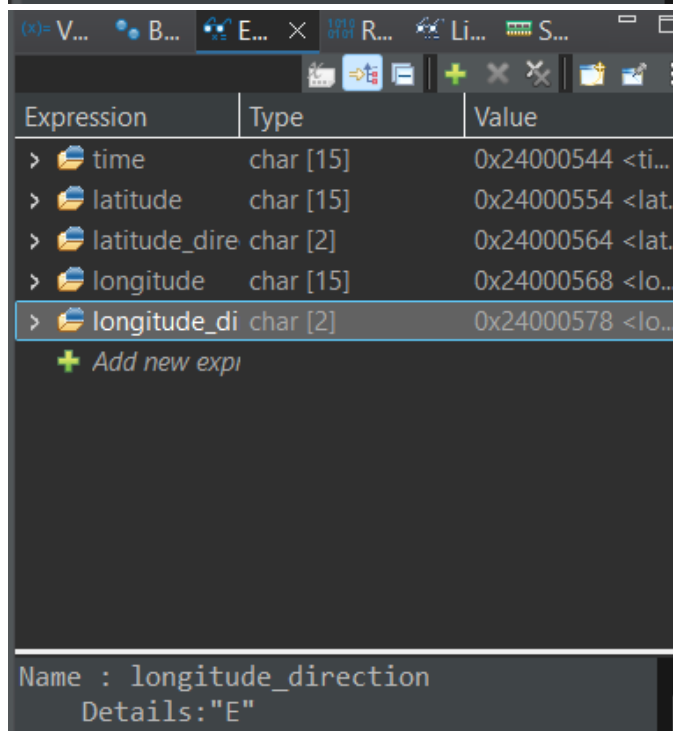


The screenshot shows a debugger window with a list of expressions. The 'longitude' variable is highlighted. Below the list, the details for the selected variable are shown.

Expression	Type	Value
> time	char [15]	0x24000544 <ti...
> latitude	char [15]	0x24000554 <lat...
> latitude_dire	char [2]	0x24000564 <lat...
> longitude	char [15]	0x24000568 <lo...
> longitude_di	char [2]	0x24000578 <lo...

+ Add new expi

Name : longitude
Details: "02211.82057\0\0\0"



The screenshot shows the same debugger window, but now the 'longitude_di' variable is highlighted. The details below show its value as 'E'.

Expression	Type	Value
> time	char [15]	0x24000544 <ti...
> latitude	char [15]	0x24000554 <lat...
> latitude_dire	char [2]	0x24000564 <lat...
> longitude	char [15]	0x24000568 <lo...
> longitude_di	char [2]	0x24000578 <lo...

+ Add new expi

Name : longitude_direction
Details: "E"

To verify the accuracy of the system, these raw NMEA coordinates (DDMM.MMMMM format) can be converted into standard decimal degrees:

- **Latitude:** 41 degrees and 44.26291 minutes North $\sim 41.7377^\circ$ N
- **Longitude:** 022 degrees and 11.82057 minutes East $\sim 22.1970^\circ$ E

When checked on a map, these precise coordinates point directly to Štip, North Macedonia. This confirms that the antenna locked onto the satellites perfectly and the embedded software parsed the continuous UART stream flawlessly without dropping or misplacing any characters.

While the parsed coordinates successfully point to Štip, I observed a slight deviation when plotting them on a map. The exact coordinates show a location slightly shifted from my actual physical address (Black dot).

